

TenantDAO – Blockchain-Based Tenant Management & Voting System

PROJECT REPORT



Bachelor of Computer Applications

Submitted By :

Enrol. No.	Name of Student	Batch
23517092	Anushka Jaiswal	BCA 3
23517074	Aakriti Kumari	BCA3

Introduction to Blockchain Technologies(21B12CS317)
EVEN, VI Semester

3rd Year

Session: 2025-26

Jaypee Institute of Information Technology, Noida

Introduction

Blockchain technology is revolutionizing the way digital transactions and organizational governance are handled. One of the most important applications of blockchain is the Decentralized Autonomous Organization (DAO), where decisions are made collectively through voting rather than by a central authority. In rental societies or tenant communities, managing shared funds can become difficult due to trust issues and lack of transparency. Usually, one person controls the funds, which can lead to misuse, corruption, or disputes. TenantDAO is a smart contract-based solution that eliminates the need for a centralized authority. All members of the tenant group participate equally in decision-making. Any member can create proposals such as maintenance payments, repairs, electricity bills, or community expenses. Members vote on these proposals, and once enough votes are received, the payment is executed automatically.

The project ensures:

- Transparency in fund usage
- Secure transactions
- Equal voting rights
- Automated execution
- Immutable transaction records

Abstract

TenantDAO is a blockchain-based decentralized application developed using Solidity smart contracts on the Ethereum blockchain. The project is designed to manage shared tenant funds in a transparent, secure, and democratic manner. Traditional tenant fund management systems often suffer from issues such as lack of transparency, centralized control, delayed approvals, and misuse of funds. To solve these problems, TenantDAO introduces a decentralized voting mechanism where all registered tenants become members of a DAO (Decentralized Autonomous Organization). Any member can create a spending proposal, and other members can vote on it. Once the proposal receives the required number of approvals (quorum), the smart contract automatically transfers funds to the specified address without requiring a middleman. The project uses Ethereum smart contracts written in Solidity and can be deployed using Remix IDE and MetaMask wallet. The system demonstrates core blockchain concepts such as decentralization, transparency, immutability, voting mechanisms, and automated fund transfers. This project is suitable for apartments, hostels, societies, and rental communities where tenants collectively manage maintenance or utility funds.

Problem Statement

In many apartments, hostels, rental houses, and shared living communities, tenants usually contribute money for common expenses such as maintenance, electricity bills, water supply, repairs, cleaning services, security charges, or other community-related activities. Traditionally, this shared fund is managed by a single person such as a landlord, building manager, or a selected tenant representative. Although this method is commonly used, it often creates several practical problems and trust issues among members.

One of the biggest problems in traditional systems is the lack of transparency. Tenants usually do not have a clear view of how the collected money is being used, where it is being spent, or whether all transactions are genuine. Since records are often maintained manually or privately, it becomes difficult for everyone to verify expenses.

Another major issue is the dependency on a single authority or individual. When one person has complete control over the funds, the entire system becomes centralized. This can lead to biased decisions, delayed approvals, or misuse of money. In some cases, disputes arise because tenants may feel excluded from important financial decisions. The traditional process is also time-consuming and inefficient. Before making any payment or approving any expense, tenants usually need to communicate through phone calls, messages, or physical meetings. Coordinating approvals from multiple people can cause delays in urgent situations such as emergency repairs or bill payments. Security and trust are also important concerns. Since transactions are handled manually or through personal accounts, there is always a risk of fraud, unauthorized spending, manipulation of records, or accidental errors. Tenants may not fully trust the system because there is no permanent and tamper-proof record of transactions.

To solve these challenges, there is a need for a secure, transparent, and decentralized solution where all tenants can participate equally in financial decisions. A blockchain-based DAO (Decentralized Autonomous Organization) system can provide an effective solution by allowing tenants to create proposals, vote collectively, and automatically execute approved payments using smart contracts.

Objectives of the Project

- To create a decentralized tenant management system.
- To enable secure voting on spending proposals.
- To automate fund transfers after approval.
- To ensure transparency and trust among tenants.
- To remove dependency on centralized authorities.
- To demonstrate practical implementation of blockchain and smart contracts

Technologies Used

1. Solidity

Solidity is the main programming language used to develop the smart contract of the TenantDAO project. It is specially designed for writing blockchain-based applications on the Ethereum network. Using Solidity, the rules of the DAO system such as proposal creation, voting, quorum checking, and automatic fund transfer are programmed directly into the smart contract. Since smart contracts are immutable after deployment, Solidity helps in creating secure and tamper-proof logic for the project.

2. Ethereum Blockchain

Ethereum is the decentralized blockchain platform on which the TenantDAO smart contract is deployed. It allows the project to operate without any central authority. All transactions, votes, and fund transfers are stored permanently on the blockchain, ensuring transparency and security. Ethereum also provides support for smart contracts, which makes automated execution of tenant decisions possible.

3. Remix IDE

Remix IDE is an online development environment used for writing, compiling, testing, and deploying Solidity smart contracts. In this project, Remix IDE was used to create and execute the TenantDAO contract. It provides an easy-to-use interface for blockchain development and helps in debugging errors during testing. Remix is especially useful for beginners because it does not require complex installation processes.

Working of the System

- **Step 1: Contract Deployment**

The smart contract is deployed with 8 member addresses.

- **Step 2: Funding the Contract**

Ether is sent to the smart contract wallet.

- **Step 3: Proposal Creation**

A member creates a proposal including:

Description

Amount

Receiver
address

- **Step 4: Voting Process**

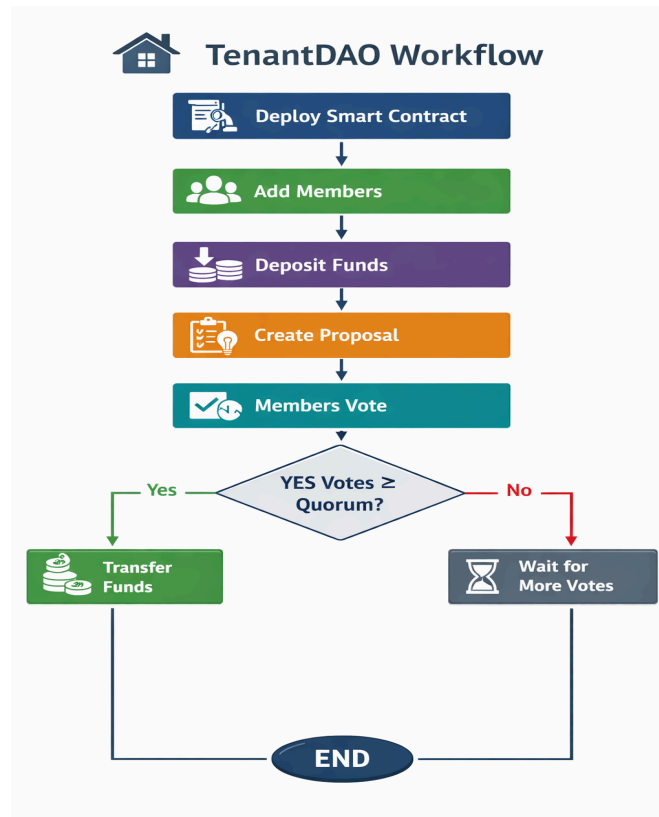
Members vote YES or NO.

- **Step 5: Quorum Check**

If YES votes become equal to or greater than the quorum (5 votes), the proposal is approved.

- **Step 6: Automatic Transfer**

Funds are automatically transferred to the receiver.



FLOW DIAGRAM OF PROJECT

Features of TenantDAO

- Decentralized governance
- Transparent transactions
- Automatic execution
- Secure blockchain storage
- No middleman required
- Immutable records
- Equal voting rights

CODE

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
```

```
contract TenantDAO {
```

```
    address[] public members;
    uint public quorum = 5;
```

```
    struct Proposal {
        string desc;
        uint amount;
        address payable to;
        uint yes;
        uint deadline;
        bool done;
        mapping(address => bool) voted;
    }
```

```
    Proposal[] public proposals;
```

```
    constructor(address[] memory _members) {
        require(_members.length == 8, "Need exactly 8 members");
        members = _members;
    }
```

```
    receive() external payable {}
```

```

// check member
function isMember(address u) internal view returns (bool) {
    for (uint i = 0; i < members.length; i++) {
        if (members[i] == u) return true;
    }
    return false;
}

// create proposal
function create(string memory d, uint amt, address payable to) public {
    require(isMember(msg.sender), "Not a member");

    Proposal storage p = proposals.push();
    p.desc = d;
    p.amount = amt;
    p.to = to;

    // increased time for demo (IMPORTANT FIX)
    p.deadline = block.timestamp + 10 minutes;
}

// vote
function vote(uint id, bool yesVote) public {
    require(id < proposals.length, "Invalid ID");

    Proposal storage p = proposals[id];

    require(isMember(msg.sender), "Not member");

```

```
require(!p.voted[msg.sender], "Already voted");
require(!p.done, "Already executed");
require(block.timestamp < p.deadline, "Voting expired");
```

```
p.voted[msg.sender] = true;
```

```
if (yesVote) {
    p.yes++;
}
```

```
// auto execute
if (p.yes >= quorum) {
    require(address(this).balance >= p.amount, "Not enough funds");
```

```
    p.done = true;
    p.to.transfer(p.amount);
}
}
```

```
// view balance
function balance() public view returns (uint) {
    return address(this).balance;
}
}
```

OUTPUT

The screenshot displays the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar is active, showing the 'Environment' section with 'Remix VM' and 'London' selected. The 'Deploy' section shows the 'TenantDAO' contract as 'Completed'. Below this, the 'Call data' and 'Parameters' fields are visible, with a 'Deploy' button at the bottom. The main editor area shows the Solidity code for the 'TenantDAO' contract, which includes a quorum of 5 and a proposal structure. The bottom panel shows the 'Explain contract' section with a list of transactions, including a successful vote transaction.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.0;
3
4 contract TenantDAO {
5
6     address[] public members;
7     uint public quorum = 5;
8
9     struct Proposal {
10         string desc;
11         uint amount;
12         address payable to;
13         uint yes;
14         uint deadline;
15         bool done;
16         mapping(address => bool) voted;
17     }
18
19     Proposal[] public proposals;
20
21     constructor(address[] memory _members) {
```

Transaction logs:

- [w] from: 0xA8B...35b2 to: TenantDAO.vote(uint256,bool) 0xA5a...fcb59 value: 0 wei data: 0xc9d...00001 logs: 0 hash: 0x856...2c069
- [w] from: 0x787...cabab to: TenantDAO.vote(uint256,bool) 0xA5a...fcb59 value: 0 wei data: 0xc9d...00001 logs: 0 hash: 0x0f8...2e166

The screenshot displays the Remix IDE interface, specifically the 'DEPLOY & RUN TRANSACTIONS' sidebar. The 'Environment' section shows 'Remix VM' and 'London'. The 'Account 1' section shows the address '0x5B3...eddC4' with a balance of '99.999 ETH'. The 'Deployed Contracts' section shows two instances of the 'TenantDAO' contract, one at '0xd91...39138' and another at '0xf8e...9fBe8', both deployed '9min ago' and '28s ago' respectively. The 'Transactions recorder' section is partially visible at the bottom.

Deployed Contracts 3

+ Add Contract



Interact with a deployed contract



TenantDAO

0xbA6...D1c55



Remix VM

12min ago



TenantDAO

0x79a...a89cc



Remix VM

12min ago



TenantDAO

0xA5a...fcb59



Remix VM

16min ago



Transactions recorder 42

Save



Contract call

Transaction list

Newest



TenantDAO

0xA5a...fcb59

TenantDAO

0xd91...39138

Remix VM

19min ago

Functions

Low level interaction

calldata

Value1ether

Gas limitauto0

Transact

Balance1.0 ETH

create string, uint256, address

Fix Limit

1000000000000000000

0x5B38Da6a701c568545dCfcB03FcB875f56beddC4

CalldataParameters

Value0wei

Gas limitauto0

Transact

proposals

uint256

0

Calldata

Parameters

0:string: desc Fix Lift

1:uint256: amount 1000000000000000000

2:address: to 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4

3:uint256: yes 0

4:uint256: deadline 1778097506

5:bool: done false

Call

Account 5

97.999 ETH

0x617...5E7f2

vote

uint256, bool

0

true

Calldata

Parameters

Value

0

ether

Gas limit

auto

0

Transact

Balance

1.0 ETH

✓

[vm] from: 0x617...5E7f2 to: TenantDAO.vote(uint256,bool) 0xA5a...fcb59 value: 0 wei data: 0xc9d...00001 logs: 0 hash: 0xe1b...dd6d1

Debug

✓

[vm] from: 0x617...5E7f2 to: TenantDAO.(constructor) value: 0 wei data: 0x608...d1ff7 logs: 0 hash: 0xe51...25c1e

Debug

✓ [vm] from: 0x617...5E7f2 to: TenantDAO.(constructor) value: 0 wei data: 0x608...diff7 logs: 0 hash: 0xee6...a9023 [Debug](#) ▾

✓ [vm] from: 0x617...5E7f2 to: TenantDAO.(receive) 0xA5a...fcb59 value: 10000000000000000000 wei data: 0x logs: 0 hash: 0x322...e99ca [Debug](#) ▾

✓ [vm] from: 0x617...5E7f2 to: TenantDAO.create(string,uint256,address) 0xA5a...fcb59 value: 0 wei data: 0x36a...00000 logs: 0 hash: 0xe71...04203 [Debug](#) ▾

CALL [call] from: 0x617f2E2fd72FD905503197092aC168c91465E7f2 to: TenantDAO.proposals(uint256) data: 0x013...00000 [Debug](#) ▴

✓ [vm] from: 0x617...5E7f2 to: TenantDAO.vote(uint256,bool) 0xA5a...fcb59 value: 0 wei data: 0xc9d...00001 logs: 0 hash: 0x01b...dd6d1 [Debug](#) ▾

✓ [vm] from: 0xA8b...35cb2 to: TenantDAO.vote(uint256,bool) 0xA5a...fcb59 value: 0 wei data: 0xc9d...00001 logs: 0 hash: 0x856...2c069 [Debug](#) ▾

✓ [vm] from: 0x787...caba8 to: TenantDAO.vote(uint256,bool) 0xA5a...fcb59 value: 0 wei data: 0xc9d...00001 logs: 0 hash: 0x0f8...2e166 [Debug](#) ▾

✓ [vm] from: 0x482...C02db to: TenantDAO.vote(uint256,bool) 0xA5a...fcb59 value: 0 wei data: 0xc9d...00001 logs: 0 hash: 0x04e...0d4cd [Debug](#) ▾

✓ [vm] from: 0x583...eddC4 to: TenantDAO.vote(uint256,bool) 0xA5a...fcb59 value: 0 wei data: 0xc9d...00001 logs: 0 hash: 0x92b...a4e56 [Debug](#) ▾

CALL [call] from: 0x5838Da6a701c568545dCfcB03Fc8875f56beddC4 to: TenantDAO.balance() data: 0xb69...ef8a8 [Debug](#) ▾

Future Scope

TenantDAO can be improved further by adding:

1. Web-based frontend dashboard
2. Dynamic member addition/removal
3. Multi-signature approvals
4. Mobile application integration
5. NFT-based tenant identity

Conclusion

TenantDAO successfully demonstrates how blockchain technology and smart contracts can be used to solve real-life problems related to trust, transparency, and fund management in tenant communities. In traditional systems, managing shared expenses often depends on one person, which can lead to confusion, delays, lack of transparency, and trust issues among tenants. This project provides a better alternative by introducing a decentralized and automated management system. Through the use of a DAO (Decentralized Autonomous Organization), all tenants become active participants in decision-making. Instead of relying on a single authority, members can collectively create proposals, vote on expenses, and approve transactions in a fair and transparent manner. Once the required number of votes is achieved, the smart contract automatically executes the payment without any manual intervention. The project successfully implements important blockchain concepts such as decentralized governance, blockchain-based voting, secure handling of shared funds, and automatic execution of approved proposals. Since all transactions are stored on the Ethereum blockchain, the system becomes transparent, secure, and tamper-proof.

Overall, TenantDAO proves that blockchain technology is not limited only to cryptocurrencies or digital payments. It can also be effectively used in community management systems, apartment societies, hostels, and other shared environments where transparency and collective decision-making are important. The project highlights the future potential of decentralized systems in creating more secure, trustworthy, and efficient management solutions.

References

1. Remix IDE Documentation
2. Ethereum Official Documentation
3. [Solidity Documentation](#)
4. [MetaMask Official Website](#)
5. Blockchain Research Papers
6. Smart Contract Tutorials and Articles
7. [Ethereum Developer Resources](#)